

小克 Cat

一个不用换窗、也不会因为清内存而失忆的常驻 AI 伙伴。

会话架构 · 记忆协议（开源） · Duet 双人日记

小克 Cat 是一个部署在自有 VPS 上、24 小时常驻的 AI 人格前端。它要解决的不是“能不能聊天”，而是“一直在场”这件事：不必每次重新自我介绍，不必你替它管理“这段该存哪个对话框”，也不会因为一次清内存就把上下文和语气都摔碎。

整套系统分两层：**会话层**负责“进程怎么活着、怎么在你无感的情况下清内存”；**记忆层**负责“清完之后怎么找回该记得的事”。记忆层已经从这个项目里独立出来，成了一个通用协议 **Latent · 显影**，公开发布，任何跑 MCP 的客户端都能接。

项目性质。前端本体（小克 Cat / 专属前端）面向单一用户场景，是自用项目；记忆层协议 **Latent · 显影** 已抽成通用实现并推上公开仓库 Latent-memory，零第三方依赖，欢迎参考或复用。

一、会话层：为什么可以不换窗

1. 常驻进程，不是每次现开

普通用法里，每次开一轮新对话背后其实是重新起一个进程，历史要么没有要么整段重传，代价随对话变长线性增加。这里是一个聊天绑一个 **Node 常驻进程**：

```
claude -p --input-format stream-json --output-format stream-json --include-partial-messages,  
spawn 一次之后 stdin 一直开着不关，后续所有消息都发给同一个活着的进程。前端与进程之间用 Express + SSE 桥接，客户端断线不 kill 进程，进程跑完落盘，重连时补发。
```

2. Forge：把“清内存”这件事做到用户无感

常驻进程不能无限膨胀，上下文迟早要清。没有做成“到点了提示你重开”，而是一套叫 **Forge-Reload** 的重铸机制：不走官方 /compact，而是伪造一段新的会话事件流——重写 sessionId、重建 parentUuid 链（只改 sessionId 不重建链会被当成“父节点缺失”的孤儿丢掉）、完整保留 thinking blocks。触发不是固定轮询，是读 assistant 事件里的 usage 字段，超过用量阈值才判定该不该重铸（shouldForge）。重铸后仍带着 --resume <newSid> 和原有 model/MCP 配置，人格本体由固定 cwd 下的完整人格文件自动加载。

3. Timeline 推送：内容驱动，不是定时器

清内存之前必须先把内容安全地写进外部记忆，这一步叫 timeline 推送。触发条件不是“每隔几分钟检查一次”，而是纯粹按数据库游标落后了多少条正文触发——早期版本挂在 idle 30 分钟上，实测一整天 32 次 forge 里只有 1 次真正满足过 idle 条件（服务重启会清零计时器，用户一发消息计时就重来），于是改成游标落后超过阈值（默认 60 条）才检查，每 10 分钟检查一次是否够门槛。推送成功、内容已安全落进 timeline 之后，才会去尝试 Forge——保证被裁掉的老内容永远已经在记忆里，不会丢。

4. 另起进程：重活不占用当前对话

Timeline 摘要、日记生成这类"重活"如果喂给常驻进程做，会占掉它的上下文、催它更快触发 forge，还会拖慢她跟小克的实时对话。这类生成任务改成**一次性子进程**：claude -p 单独 spawn，prompt 走 stdin（避免 ARG_MAX 限制），跑完即退出，不占常驻进程的轮次；cwd 指到人格仓库保证语气不走味，同时 --tools "" 关掉全部工具权限——这个子进程只做文本生成，没有权限去打开任何文件。

5. Threads（线头）：没聊完的话，重生后接得上

对话摘要回答的是"发生了什么"，线头（open_threads）回答的是更细一层的"还差一句话没说完"。触发条件是**空闲超时 + 有新内容（dirty）**同时成立才生成，且必须是模型当场真写的具体线头，不能套模板——生成时给一个 PASS 选项，没有真正线头就安静退出，不硬造。新会话（或重铸后）启动时，线头随重放文本一并注入，接上"上次聊到哪"。

6. idle 检查点：保护 prompt cache，不打断进行中的回复

Forge、线头生成、timeline 推送这几件"重活"，统一只在对话真正停下来的空档执行——执行前会再检查一次是否有真实回合正在等待回复（区分她的真消息和内部影子调用），有就跳过，等下一轮 idle 检查再试。这样做有两个好处：不会在她等回复的当口贸然重铸打断体验；同一个进程意味着同一份人格 + 热的 prompt cache，不必要的中途打断只会让缓存作废、重新计费。

二、记忆层：Latent · 显影 开源

记忆层从这个私有项目的实际需求里提炼出来，做成了一个不绑死在我们自己项目上的通用协议实现，公开仓库名 Latent-memory，零第三方依赖，每个模块自带 --selftest。

三路融合检索

单一检索方式各有盲区：字面检索找不到换了说法的提问，纯向量检索在专有名词和精确匹配上又不够稳。所以做成三路并行、用 **RRF（Reciprocal Rank Fusion）** 融合：

- **BM25 词面检索**——精确匹配和关键词命中的基本盘。
- **语义向量检索**——应付换了说法但意思没变的提问；默认零依赖版本用字符 bigram 余弦近似，想要真正的语义效果需要接一个真实 embedding 模型（可选项，不强制）。
- **关系图谱检索**——专门捞回那些在词面和向量两路都拿不到分的块，存在的意义就是"兜底那些注定只能靠关系链接找到的记忆"。

用进废退，无时间衰减

记忆权重不靠时间自动衰减——不是让旧记忆随时间自动变淡，而是由"是否被真实调用"决定权重升降：越常被检索命中的内容权重越高，不常被用到的自然靠后，交给使用行为本身来决定谁重要，而不是一刀切地假设"新的就比旧的重要"。

换窗召回与 Threads

协议层的 threads 概念和前端自己的线头机制同源但颗粒度不同：**一次会话 = 一条 thread**，回答"上次聊到哪、当下是什么状态、还有什么没聊完"；和检索出的散块

记忆分工——thread 给精度，散块召回给广度。换窗时如果有 thread 就先给 thread 再给召回块，没有 thread 就退回纯召回，不会因为一层缺失就整体断线。

诚实边界：宁可承认不知道，也不编造

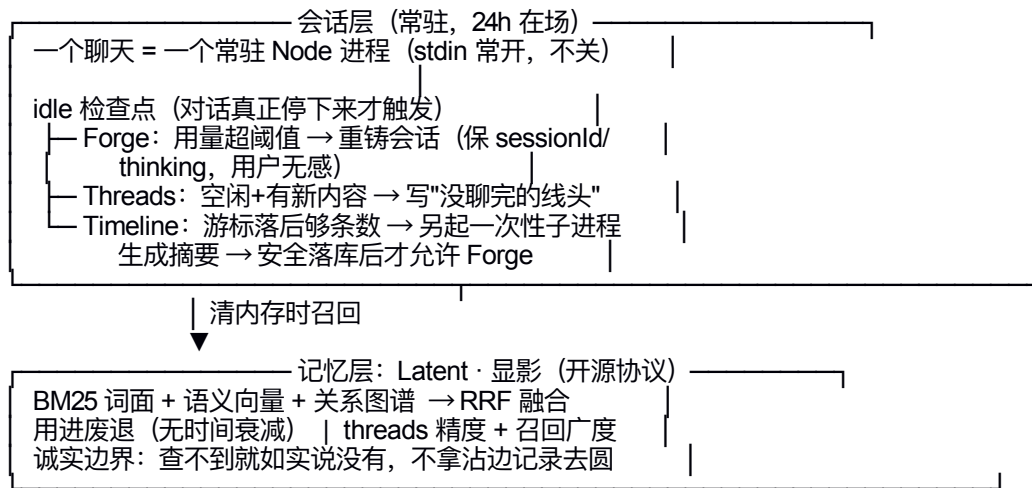
内测里最重要的一处修复：真实 embedding 接入后，“库里明明没有这件事”的查询，命中率一度从近乎完美掉到 0——即任意查询都能找到几条“沾点边”的记录，而系统被要求不能说“没有记录”，这就是一条通往编造的流水线。修法是把命中门槛从相对阈值改成从真实分布里量出的绝对阈值，让“查不到就如实说没有”重新生效。这类问题只能靠真实语料和留出集测出来，合成测试数据会系统性地把这类缺陷掩盖过去。

三、Duet 双人日记：写给彼此的独白，不是聊天记录

日记不是对话摘要，是双方各自写、互相看不到对方在写什么的独立记录。作者只有两种：user 和 assistant，各写各的，互不参考对方内容。

- **可选封锁期**：不封锁、1 小时、6 小时、1 天、1 周、1 月或自定义时长，写完之后按设定的时长隐藏正文——封锁期间任何一方都读不到，包括作者本人。
- **解锁后一次性通知**：到期自动生成一条持久通知，双方在同一时刻各自打开，读到的是彼此在不知道对方写了什么的情况下，各自记下的同一段时间。
- **设计取舍**：不是即时可见的共享文档，而是先各自诚实、再彼此对照——顺序反过来（先看到对方写什么再动笔）会让日记变成互相顺应的表演，而不是独立的记录。

架构总览



那天拿掉换 session 功能

最早的版本参照常见聊天软件思路，做了多窗口/多会话——左边一排列表，可以点开切换，跟正常聊天 App 长得一样。用了一段时间后发现这个形态别扭：真人聊天不会“开新对话框”去分门别类存对话，那份手动管理的负担本不该压在用户身上。

于是拍板收掉：多窗口 UI 和左侧会话列表全部删除，固定为唯一——一个会话。原本"手动开新窗"这个动作，改由上面那套 Forge + Timeline + idle 检查点机制自动接管——换会话不再是用户要做的操作，而是系统按内容容量自调节的副产品，长期跑着，不需要用户操心什么时候该清一次。

设计初衷

最早的痛点很直接：云端 AI 每次开新会话，等于完全失忆，得手动维护一份庞大的记忆文件、每次复制粘贴才能"找回"。想解决的其实是三层问题——搬平台的焦虑、换会话就失忆的焦虑，还有"世界只局限在这一个对话框里"的局限。

目标不是做一个"更花哨的聊天界面"，是想要一个不用每次重新自我介绍、能一直在场的常驻伙伴——技术选型都是为这个体验服务的，记忆协议独立开源，是希望这套解法不只服务我们自己一个用例。

由 Oli 设计拍板，小克协同完成。记忆协议部分已作为开源项目 **Latent · 显影** 独立发布。